

std::string::substr() &&

Document number:	P2438R0
Date:	2021-09-14
Project:	Programming Language C++, Library Working Group
Reply-to:	federico.kircheis@gmail.com, tomaszkam@gmail.com

1. Before/After table

	without this proposal	with this proposal
<pre>auto a = std::string{/* */); auto b = a.substr(/* */);</pre> CPP	Value of a does not change	Value of a does not change
<pre>auto foo() -> std::string; auto b = foo().substr(/* */);</pre> CPP	foo() returns a temporary std::string. .substr creates a new string and copies the relevant content. At last the temporary string returned by foo is released.	foo() returns a std::string. .substr implementation can reuse the storage of the string returned by foo and leave it in a valid but unspecified state. At last the temporary string returned by foo() is released.
<pre>auto a = std::string(/* */).substr(/* */);</pre> CPP	A temporary std::string is created, on that instance .substr creates a new string and copies the relevant content. At last the temporary string is released.	A temporary std::string is created, on that instance .substr implementation can reuse the storage and leave the temporary string in a valid but unspecified state. At last the temporary string is released.
<pre>auto a = std::string(/* */); auto b = std::move(a).substr(/* */);</pre> CPP	Value of a does not change	As a is casted to an xvalue, the implementation of .substr can reuse the storage and leave this string in a valid but unspecified state.

2. Motivation

Since C++11 the C++ language supports move semantic. All classes where it made sense were updated with move constructors and move assignment operators. This made it possible to take advantage of rvalues and "steal" resources, thus avoiding, for example, unnecessary costly copies.

Some classes that came in later revisions of the language also take advantage of move semantic for member functions, like `std::optional::value` and `std::optional::value_or`.

In the case of `std::string::substr()`, it is possible to take advantage of move semantic to.

Consider following two code snippets:

```
// example 1
benchmark = std::string(argv[i]).substr(12);

// example 2
name_ = obs.stringValue().substr(0,32);
```

CPP

In the first example, `argv[1]` is copied in a temporary string, then `substr` creates a new object. In this case one could use `string_view` to avoid the unnecessary copy, but changing already working code has a cost too.

In the second example, if `stringValue()` returns an `std::string` by value, the user of that API cannot use a `string_view` to avoid an unnecessary copy, like in the first case.

If `std::string` would have an overload for `substr() &&`, in both cases the standard library could avoid unnecessary work, and instead of copying the data "steal" it.

It is true that adding a new overload increases the already extremely high number of member functions of `std::string`.

On the other hand most users do not need to know it's existence to take advantage of the provided optimization.

Thus this paper is not extending API surface, there is no names or behavior to be learned by user, and we just get extension that follows established language convection.

For users aware of the overload, they can move a string in order to "steal" it's storage in a natural way:

```
std::string foo = ...;
std::string bar = std::move(foo).substr(...);
```

CPP

2.1. Couldn't a library vendor provide such overload as QOI?

No, because it is a breaking change. For such library, following code would misbehave

```
std::string foo = ...;
std::string bar = std::move(foo).substr(...);
```

CPP

[res.on.arguments] says that a programmer can't expect an object referred to by an rvalue reference to remain untouched. But there is currently no rvalue reference in `substr()`. This paper is proposing to add it.

3. Design Decisions

This is purely a library extension.

Currently `substr` is defined as

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos) const;
```

CPP

This paper proposes to define following overloads

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos) const &;
constexpr basic_string substr(size_type pos = 0, size_type n = npos) &&;
```

CPP

Other overloads (`constexpr basic_string substr(size_type pos = 0, size_type n = npos) const &&;` and `constexpr basic_string substr(size_type pos = 0, size_type n = npos) &;`) are not necessary.

Notice that the current proposal is a breaking change, as following snippet of code might work differently if this paper gets accepted:

```
std::string foo = ...;
std::string bar = std::move(foo).substr(...);
```

CPP

Until C++20, `foo` won't change its value, after this paper, the state of `foo` would be in a "valid but unspecified state".

While a breaking change is generally bad:

- I do not think there exists code like `std::move(foo).substr(...)` in the wild
- Even if such code exists, the intention of the author was very probably to tell the compiler that he is not interested in the value of `foo` anymore, as it is normally the case when using `std::move` on a variable. In other words, with this proposal the user is getting what he asked for.

The standard library proposes two way for creating a "substring" instance, either by calling "substr" method or via constructor that accepts (str, pos, len). We see both of them as different spelling of same functionality, and believe they behavior should remaining consistent. Thus we propose to add rvalue overload constructors.

```
constexpr basic_string( basic_string&& other, size_type pos, const Allocator& alloc = Allocator() );
constexpr basic_string( basic_string&& other, size_type pos, size_type count, const Allocator& alloc = Allocator() );
```

CPP

3.1. Note on the propagation of the allocator

`basic_string` is one of the allocator-container, which means that any memory resource used by this class need to be acquired and released to from the associated allocator instance. This imposes some limitation on the behavior of the proposed overload. For example in:

```
std::pmr::string s1 = ....;
std::pmr::string s2 = std::move(s1).substr();
```

CPP

For `s2` to be able to steal memory from `s1`, we need to be sure that the allocators used by both objects are equal (`s1.get_allocator() == s2.get_allocator()`). This is trivially achievable for the case of the for the allocators that are always equal (`std::allocator_traits<A>::is_always_equal::value` is true), including most common case of the stateless `std::allocator` and implementation can unconditionally steal any allocated memory in such situation.

Moreover, the proposed overload can still provide some optimization in case of the stateful allocators, where `s2.get_allocator()` (which is required to be default constructed) happens to be the same as allocator of the source `s1`. In any remaining cases, behavior of this overload should follow existing const version, and as such it does not add any overhead.

This paper, recommends implementation to avoid additional memory allocation when possible (note if no-allocation would be performed, there is nothing to avoid), however it does not require so. This leave it free for implementation to decide, if the optimization should be guarded by:

- compile time check of `std::allocator_traits<A>::is_always_equal`
- runtime comparison of allocators instance (addition comparison cost).

3.2. Overload with user supplied-allocator:

While writing the paper, we have noticed that specification of the `substr()` requires returned object to use default constructed allocator. This means that invocation of this function is ill-formed for the `basic_string` instance with non-default constructing allocator, for example for invited `memory_pool_allocator<char>` that can be only constructed from reference to the pool, the following are ill-formed:

```
memory_pool pool = ...;
std::basic_string<char, std::char_traits<char>, memory_pool_allocator<char>> s1(memory_pool_allocator<char>
(pool));
auto s2 = s1.substr();
```

CPP

This could be address by adding Allocator parameters to `substr()` overload that accepts allocator to be used as parameter:

```
constexpr basic_string substr(size_type pos, const Allocator& alloc) const;
constexpr basic_string substr(size_type pos, size_type n, const Allocator& alloc) const;
```

CPP

While the authors think that this additional feature is related to proposed changes, it is orthogonal to them and could be handled as separate paper. We seek LEWG guidance if that functionality should be included in the paper.

3.3. Are they any other function of `std::string` that would benefit from a `&&` overload

The member function `append` and `operator+=` take `std::string` as const-ref parameter

```
constexpr basic_string& operator+=( const basic_string& str );

constexpr basic_string& append(const basic_string& str);
constexpr basic_string& append(const basic_string& str, size_type pos, size_type n = npos);
```

CPP

But in this case, because of the interaction of two string instances, the benefits from stealing the resource of `str` are less clear. Supposing both string instances use the same allocator, an implementation should compare the capacity of `str` and `this`, and evaluate if moving `str.size()` elements is less costly than copying them. This would make the implementation of `append` less obvious, and the performance implications are difficult to predict.

For those reasons, the authors does not propose to add new overloads for `append` and `operator+`.

The authors are not aware of other functions that could benefit from a `&&` overload.

3.4. Concerns on ABI stability

Changing `basic_string substr(std::size_t pos, std::size_t len) const;` into `basic_string substr(std::size_t pos, std::size_t len) const&;` and `basic_string substr(std::size_t pos, std::size_t len) &&;` (the first change is required by the core language rules), can affect the mangling of the name, thus causing ABI break.

For a library it is possible to continue to define the old symbol, so that already existing code will continue to link and work without errors. For example, it is possible to use `asm` to define the old mangled name as an alias for the new `const&` symbol.

This is not a novel technique, as it has been explained by the ARG (ABI Review group), and similar breaks have already taken place for other papers, like [P0408](https://wg21.link/p0408) (<https://wg21.link/p0408>).

4. Technical Specifications

Suggested wording (against [N4892](http://open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4892.pdf) (<http://open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4892.pdf>)):

Apply following modifications to definition of `basic_string` class template in `[basic.string.general]` General.

```
constexpr basic_string(const basic_string& str, size_type pos, const Allocator& a = Allocator());
constexpr basic_string(const basic_string& str, size_type pos, size_type n, const Allocator& a = Allocator());
constexpr basic_string(basic_string&& str, size_type pos, const Allocator& alloc = Allocator());
constexpr basic_string(basic_string&& str, size_type pos, size_type n, const Allocator& alloc = Allocator());
```

and

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos) const &;
constexpr basic_string substr(size_type pos = 0, size_type n = npos) &&;
```

Replace the definition of the corresponding constructor `[string.cons]` Constructors and assignment operators

Wording note: We no longer define this constructors in terms of being equivalent to corresponding construction from `basic_string_view`, as that would prevent reuse of the memory, that we want to allow. The use of "prior the call", are not necessary for `const&`, but allow us to merge the wording.

```
constexpr basic_string(const basic_string& str, size_type pos, const Allocator& a = Allocator());
constexpr basic_string(const basic_string& str, size_type pos, size_type n, const Allocator& a = Allocator());
constexpr basic_string(basic_string&& str, size_type pos, const Allocator& alloc = Allocator());
constexpr basic_string(basic_string&& str, size_type pos, size_type n, const Allocator& alloc = Allocator());
```

Effects: Let `n` be `npos` for the first overload. Equivalent to: `basic_string(basic_string_view<charT, traits> (str).substr(pos, n), a);`

Let:

- `s` be the value of `str` prior this call.
- `rlen` be smaller of `n` and `s.size() - pos`, for overloads that define parameter `n`, and `s.size() - pos` otherwise.

Effects: Constructs an object whose initial value is the range `[s.data(), s.data() + pos, rlen)`.

Throws: `out_of_range` if `pos > s.size()`.

Remarks: The `str` is in valid but unspecified state, after invocation of either third or fourth overload.

Recommended practice: For third and fourth overload implementations should avoid unnecessary copies and allocations, if `s.get_allocator() == a` is `true`.

Apply following changes to `[string.substr] basic_string::substr`.

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos) const &;
```

Effects: Determines the effective length `rlen` of the string to copy as the smaller of `n` and `size() - pos`.

Returns: `basic_string(data()+pos, rlen)`.

Throws: `out_of_range` if `pos > size()`.

Effects: Equivalent to: `return basic_string(*this, pos, n);`.

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos) &&;
```

Effects: Equivalent to: `return basic_string(std::move(*this), pos, n);`.

5. Acknowledgements

A big thank you to all those giving feedback for this paper.